PROBLEM 1 : (***Types and values? (24 points) (Estimate: 12 minutes)***)

Consider the following variables and their values for the table below.

```
tenting = ["campout", "roll_call", "line_monitor", "game_wristband", "rivalry_game"]
```

List in the table the type of variable and its value after being assigned the expression. The first one is done for you.

variable = expression	**Type**	**Value**
`a = "sandwich"`	string	`"sandwich"`
`b = len(tenting[0:2])`	int	2
`c = len(tenting[-3]) # Negative index`	int	1
`d = tenting[0][3] * 3`		
`e = tenting[0:2] + tenting[4:]`	list	['...','...','...']
`f = "black" + " tent7"` # Notice whitespace	str	black tent7
`g = "kville"[1:]  # No number after colon`	str	ville
`h = "white_tent3".split("e")`	str	whit_tnt3
`i = "+".join(["blue", "tent", "2"])`	str	+bluetent2
`j = 2 ** 3 + 4.0`	float	12.0
`k = 4 <= 1 or 3 == 6`	boolean	false
`l = not(1>=9)`	boolean	true
`m = 1 == 1 and 2 != 5`	boolean	true

PROBLEM 2 : (*Short Code (16 pts) (Estimate: 8 minutes)*)

For Parts A and B, build a Python expression that sets `result` to the specified value.

Rules:

- Your expression must derive the result from the provided variable. Do not create new strings that replace the need to use the variable.
- Use only indexing, slicing, concatenation, `join`, and/or `split`.
- Write one single Python statement. Multi-line solutions may receive partial credit.

Example

Use `phrase` to set `result` to the string `"by"`.

```
phrase = "bicycle"
result = phrase[0] + phrase[3]
```

(Notice how this answer does not simply assign the string `result = "by"`.)

Part A (4 points)

Use `grammy_winners` to set `result` to the list `[" quincy_jones", " chick_corea"]`. Notice the whitespace in front of both strings.

```
grammy_winners = "beyonce, georg_solti, quincy_jones, chick_corea"

result =
```

Part B (4 points)

Use `name` to set `result` to the string `"lamar_kendrick"`.

```
name = "kendrick_lamar"

result =
```

For problems C and D, write the output of the code.

Part C (4 points)

```
miyazaki_films = ["totoro", "princess mononoke"]

x = miyazaki_films

x.append("spirited away")

miyazaki_films.append("kiki")

print(miyazaki_films)
```

Output:

Part D (4 points)

```
miyazaki_films = ["totoro", "princess mononoke"]

x = miyazaki_films

y = x + ["ponyo"]

print(miyazaki_films)
```

Output:

PROBLEM 3 : (*Spot the Bug (15 points) (Estimate: 15 minutes)*)

Consider a student roster where each person is stored as a 2-item list in the format `[first_name, last_name]`. The roster is a list of these lists. The following function is intended to capitalize the first letter of each person's first and last name, then return the updated list. However, the function does not work correctly. Review the code below:

```
01   def capitalize_roster(roster):
02      for student in roster:
03         first = student[0]
04         first = first[0].upper() + first[1:] # Remember to use your reference sheet
05
06         last = student[1]
07         last = last[0].upper() + last[1:]
08      return roster
09
10
11   if __name__ == "__main__":
12      students = [["ana", "reed"],["li", "nguyen"],["omar", "patel"]]
13      print(capitalize_roster(students))
```

Assuming the code worked correctly, the expected output of this code is:

```
[["Ana", "Reed"], ["Li", "Nguyen"], ["Omar", "Patel"]]
```

a) What line number contains the function call?

b) What line number contains the function definition header?

c) What list values were passed as an argument to the function call?

d) What is the variable name of the parameter?

e) What is the value of the `for` loop variable `student` in the buggy code when the first iteration of the loop finished executing?

f) What is the value of the variable `first` in the buggy code when the first iteration of the loop finished executing?

g) What is the value of the `for` loop variable `student` in the buggy code when the last iteration of the loop finished executing?

h) What is the actual output (if any) when this buggy program is executed? If an error is displayed, name or describe the error. Recall that output is what is shown in the Python console window.

i) Using the space below, rewrite the function so that it correctly capitalizes each name.

PROBLEM 4 : (*Count Special Numbers (12 points) (Estimate: 10 minutes)*)

Write a function named `count_special_numbers` that takes one parameter, `nums`, which is a list of integers. A number is considered special if it is divisible by both 2 and 3. Your function should count how many special numbers appear in the list and return that value. Assume the list may contain positive integers, negative integers, and the integer zero.

Here are several examples of calls to this function.

Function Call	**Return Value**	**Explanation**
`count_special_numbers([0, 3, 6, 4])`	`2`	Only 0 and 6 are divisible by both 2 and 3. Note that 0 is divisible by any number.
`count_special_numbers([1, 5, 8])`	`0`	None of the numbers meet the condition.
`count_special_numbers([1, -6, 9])`	`1`	Only -6 is divisible by both 2 and 3.

Complete the function below.

```
def count_special_numbers(nums):
```

PROBLEM 5 : (*Validate a Duke NetID (16 points) (Estimate: 10 minutes)*)

Duke NetIDs must follow specific formatting rules. Write a function named `is_valid_netid` that takes two parameters, `full_name` and `netid`. The `full_name` parameter is a single string containing the student's first and last name separated by a single space.

A NetID is considered valid if:

- It begins with the first letter of the student's first name followed by the first letter of their last name.
- Every character after the first two must be a digit (0-9).

Return `True` if the NetID is valid. Otherwise, return `False`.

Assume the following:

- `netid` will always have a length of 4.
- `netid` will contain only letters and digits (0-9).
- All names will be in lowercase with no middle names.

Here are several examples of calls to this function.

Function Call	**Return value**	**Explanation**
`is_valid_netid("alice brown", "ab23")`	`True`	The first two letters match the initials of their name and the remaining characters are digits.
`is_valid_netid("evan taylor", "etA2")`	`False`	The third character must be a digit.
`is_valid_netid("noah king", "no16")`	`False`	Incorrect initials.

Complete the function below.

```
def is_valid_netid(full_name, netid):
```

PROBLEM 6 : (***Parking Garage Fees (16 points) (Estimate: 12 minutes)***)

A parking garage charges drivers based on how long they stay. Write a function named `parking_fees` that takes a list of positive integers named `hours`. Each integer represents the number of hours a car was parked. Return a list containing the fee for each stay using the pricing rules below:

- Parking for fewer than 2 hours costs $5.
- Parking for at least 2 hours but fewer than 5 hours costs $10.
- Parking for at least 5 hours but fewer than 10 hours costs $15.
- Parking for 10 or more hours costs $20.

Return the list of fees in the same order as the original list.

Here are several examples of calls to this function.

Function Call	**Return Value**	**Explanation**
`parking_fees([1, 3, 6, 12])`	`[5, 10, 15, 20]`	1 hour costs $5. 3 hours falls in the 2–4 hour range for $10. 6 hours falls in the 5–9 hour range for $15. 12 hours is 10 or more, so it costs $20.
`parking_fees([10, 2, 4])`	`[20, 10, 10]`	10 hours is 10 or more, so it costs $20. Both 2 and 4 hours fall in the 2–4 hour range for $10.

Complete the function below.

```
def parking_fees(hours):
```